Dairy Management System — Full Implementation Plan

Stack: Express + MySQL (`mysql2/promise`) backend, React 19 + Vite + Tailwind frontend. This plan is written so any dev can pick up a section and implement it without re-deriving the approach.

PHASE 0 — Critical Bug Fixes (do first, ~half day)

0.1 `customers` schema/frontend mismatch

`schema.sql` has no `default_quantity`/`milk_type` on `customers` (moved to `customer_requirements` + `products` in `migrate_products.js`). But `RouteAssignment.jsx` and `DailyLogs.jsx` still read those dead fields.

Fix — `RouteAssignment.jsx`:

```jsx
// before
<p className="text-xs font-semibold text-[#1551b5] mt-1">{customer.default_quantity}L {customer.milk_type}</p>
// after — use the field adminController.getCustomers already returns
<p className="text-xs font-semibold text-[#1551b5] mt-1">{customer.aggregated_requirements || 'No active requirement'}</p>
```

Fix — `adminController.getDailyLogs`: the SQL never selects quantity info at all. Add the same correlated subquery pattern used in `getCustomers`:

```js
const [logs] = await pool.query(`
  SELECT
    d.id,
    cu.name as customer_name,
    (SELECT GROUP_CONCAT(CONCAT(cr.quantity, ' ', p.unit, ' ', p.name) SEPARATOR ', ')
     FROM customer_requirements cr JOIN products p ON cr.product_id = p.id
     WHERE cr.customer_id = c.id) as aggregated_requirements,
    du.name as delivery_boy_name,
    d.status,
    d.delivered_quantity
  FROM deliveries d
  JOIN customers c ON d.customer_id = c.id
  JOIN users cu ON c.user_id = cu.id
  JOIN delivery_boys db ON d.delivery_boy_id = db.id
  JOIN users du ON db.user_id = du.id
  WHERE d.date = ?
`, [date]);
```

Fix — `DailyLogs.jsx` mapping:

```js
req: log.aggregated_requirements || 'No active requirement',
```

0.2 Delivery status lost on refresh

`DeliveryDashboard.jsx` hardcodes `status: 'Pending'` for every stop regardless of what's already in `deliveries` for today.

Fix — `deliveryController.getTodayRoute`: LEFT JOIN today's delivery record.

```
js
const [route] = await pool.query(`
  SELECT
    c.id as customer_id, u.name as customer, c.address,
    (SELECT GROUP_CONCAT(CONCAT(cr.quantity,' ',p.unit,' ',p.name) SEPARATOR ', ')
     FROM customer_requirements cr JOIN products p ON cr.product_id = p.id
     WHERE cr.customer_id = c.id) as quantity,
    del.status as delivery_status
  FROM delivery_routes dr
  JOIN customers c ON dr.customer_id = c.id
  JOIN users u ON c.user_id = u.id
  LEFT JOIN deliveries del ON del.customer_id = c.id AND del.date = CURDATE() AND del.delivery_boy_id = dr.delivery_boy_id
  WHERE dr.delivery_boy_id = ?
`, [dbId]);
```

Fix — `DeliveryDashboard.jsx` mapping:

```
js
status: item.delivery_status === 'DELIVERED' ? 'Delivered'
    : item.delivery_status === 'NOT_AVAILABLE' ? 'Missed'
    : 'Pending'
```

Also fix `markDelivered` to `UPDATE ... ON DUPLICATE KEY` instead of blind `INSERT` (currently re-toggling creates duplicate rows for the same customer/date) — add a `UNIQUE(customer_id, delivery_boy_id, date)` constraint on `deliveries` and use:

```
js
await pool.query(`
  INSERT INTO deliveries (customer_id, delivery_boy_id, date, assigned_quantity, delivered_quantity, status)
  VALUES (?, ?, CURDATE(), ?, ?, ?)
  ON DUPLICATE KEY UPDATE delivered_quantity = VALUES(delivered_quantity), status = VALUES(status)
`, [customerId, dbId, quantity, delivered ? quantity : 0, status]);
```

0.3 Hardcoded DB credentials

`seed_delivery_boys.js` and `migrate_products.js` have `password: 'Prxthvi@100'` committed in plaintext. Rewrite both to match `migrate_onetime.js`'s pattern (`require('dotenv').config()` + `process.env.DB_PASSWORD`). Rotate that DB password once removed from history (`git filter-repo` or BFG if already pushed).

0.4 Plaintext password login fallback

`authController.login` has:

```
js
if (password === user.password) { isMatch = true; }
```

This is a security hole — remove it once `seed.sql`'s raw `'123'` passwords are re-hashed. One-time migration script:

```
js
```

```
const [users] = await pool.query('SELECT id, password FROM users');
for (const u of users) {
 if (u.password.length < 30) { // not bcrypt-shaped
  const hashed = await bcrypt.hash(u.password, 10);
  await pool.query('UPDATE users SET password = ? WHERE id = ?', [hashed, u.id]);
 }
}
```

Then delete the plaintext branch in `login`.

PHASE 1 — Core Feature Completion

1.1 Products CRUD

New `productController.js`:

```
js
const list = async (req,res) => { const [rows] = await pool.query('SELECT * FROM products'); res.json(rows); };
const create = async (req,res) => {
 const { name, unit, price_per_unit, icon, in_stock } = req.body;
 await pool.query('INSERT INTO products (name, unit, price_per_unit, icon, in_stock) VALUES (?,?,?,?,?)',
  [name, unit, price_per_unit, icon || '📦', in_stock ?? true]);
 res.status(201).json({ message: 'Product created' });
};
const update = async (req,res) => {
 const { id } = req.params;
 const { name, unit, price_per_unit, icon, in_stock } = req.body;
 await pool.query('UPDATE products SET name=?, unit=?, price_per_unit=?, icon=?, in_stock=? WHERE id=?',
  [name, unit, price_per_unit, icon, in_stock, id]);
 res.json({ message: 'Product updated' });
};
const remove = async (req,res) => { await pool.query('DELETE FROM products WHERE id=?', [req.params.id]); res.json({message:'Deleted'}); };
module.exports = { list, create, update, remove };
```

Routes in `adminRoutes.js`: `GET/POST /products`, `PUT/DELETE /products/:id`. Build `Products.jsx` admin page modeled on `StaffManagement.jsx` (modal form + card grid), route it in `App.jsx` under `/admin/products`.

1.2 Inventory — deferred, sidebar link stays dead for now

Decision: not building real stock tracking in this pass. `inventory` table stays unused. Leave the `AdminLayout.jsx` menu item as-is (dead link) rather than spend time on it now — revisit post-launch if stock-out tracking becomes a real ask.

1.3 Trim/complete admin sidebar

`AdminLayout.jsx` menu has 13 items; only 5 are routed in `App.jsx`. Either scaffold thin pages for Deliveries/Customers/Payments/Reports/Branches/Settings, or delete unrouted `<Link>` entries so nothing 404s. Recommend: build **Customers** page first (uses existing `getCustomers` endpoint, just needs a table UI) since it's the most requested-looking gap.

1.4 Collect Payment (delivery boy)

`deliveryController.js` — add:

```
js
```

```
const collectPayment = async (req, res) => {
 const { customerId, amount, method } = req.body; // method: CASH/UPI
 await pool.query(`UPDATE bills SET status='PAID' WHERE customer_id=? AND status='PENDING' ORDER BY id LIMIT 1`, [customerId]);
 res.json({ message: 'Payment recorded' });
};
```

Route: `POST /delivery/collect-payment`. Wire into the existing "Collect Payment" quick-action card in `DeliveryDashboard.jsx`.

1.5 Real dashboard stats

`adminController.getDashboard` — replace hardcoded `todayMilk`/`delivered`:

```
js
const [[{ todayMilk }]] = await pool.query(
 `SELECT COALESCE(SUM(delivered_quantity),0) as todayMilk FROM deliveries WHERE date = CURDATE()`);
const [[{ delivered }]] = await pool.query(
 `SELECT COUNT(*) as delivered FROM deliveries WHERE date = CURDATE() AND status='DELIVERED'`);
```

PHASE 2 — Security & Config Hardening

2.1 Centralize API base URL

`frontend/.env`: `VITE_API_URL=http://localhost:5000/api` Every `fetch('http://localhost:5000/api/...')` → `fetch(\`${import.meta.env.VITE_API_URL}/...\`)`, or better, do 2.2.

2.2 Axios client with interceptor

`src/api/client.js`:

```
js
import axios from 'axios';
const api = axios.create({ baseURL: import.meta.env.VITE_API_URL });
api.interceptors.request.use(config => {
 const token = localStorage.getItem('token');
 if (token) config.headers.Authorization = `Bearer ${token}`;
 return config;
});
api.interceptors.response.use(
 res => res,
 err => {
  if (err.response?.status === 401) {
   localStorage.removeItem('token');
   window.location.href = '/login';
  }
  return Promise.reject(err);
 }
);
export default api;
```

Replace every `fetch(...)` call across `.jsx` files with `api.get('/customer/dashboard')` etc. — mechanical refactor, do file-by-file, ~25 call sites.

2.3 CORS lockdown

```
```

```
app.use(cors({ origin: process.env.FRONTEND_URL, credentials: true }));
```

2.4 Input validation

Add `zod` schemas per route, validated via middleware:

```
const { z } = require('zod');
const validate = (schema) => (req, res, next) => {
 const result = schema.safeParse(req.body);
 if (!result.success) return res.status(400).json({ message: result.error.errors[0].message });
 req.body = result.data;
 next();
};
// example
const loginSchema = z.object({ email: z.string().email(), password: z.string().min(1) });
router.post('/login', validate(loginSchema), login);
```

Apply to every POST/PUT in `authRoutes`, `adminRoutes`, `customerRoutes`, `deliveryRoutes`.

2.5 Rate limiting

```
const rateLimit = require('express-rate-limit');
const authLimiter = rateLimit({ windowMs: 15*60*1000, max: 10, message: { message: 'Too many attempts, try later' } });
router.post('/login', authLimiter, validate(loginSchema), login);
router.post('/signup', authLimiter, validate(signupSchema), signup);
```

PHASE 3 — Data Integrity

3.1 Transactions for multi-step writes

Apply the pattern already used in `checkoutOnetime` to `signup` (user + customer + default requirement insert), `addCustomer` (user + customer insert), `generateBills` (loop of inserts/updates):

```
const connection = await pool.getConnection();
try {
 await connection.beginTransaction();
 // ...queries using connection.query instead of pool.query
 await connection.commit();
} catch (err) {
 await connection.rollback();
 throw err;
} finally {
 connection.release();
}
```

3.2 FK cascade audit

`bills` and `orders`/`order_items` currently have no `ON DELETE CASCADE` back to `customers`. Migration:

```sql
ALTER TABLE bills DROP FOREIGN KEY bills_ibfk_1; -- adjust name via SHOW CREATE TABLE
ALTER TABLE bills ADD CONSTRAINT bills_customer_fk FOREIGN KEY (customer_id) REFERENCES customers(id) ON DELETE CASCADE;
```

(`orders`/`order_items` already have cascade per `migrate_onetime.js` — verify, no change needed there.)

3.3 Real migrations

Move off ad-hoc `migrate_*.js` scripts to `knex` migrations (or `db-migrate`) so schema changes are versioned and repeatable in CI/staging, not manually run once per environment.

PHASE 4 — UX / Polish

4.1 Auth context

`src/context/AuthContext.jsx` — wraps `<App />`, exposes `{ user, role, login(token, role), logout() }`, reads/writes `localStorage` in one place. Replaces manual `localStorage.getItem('role')` checks scattered in `Login.jsx`, layouts.

4.2 Toasts

`react-hot-toast` — wrap root in `<Toaster />`, replace every `alert(...)` (`CustomerDashboard.jsx` support click, `CheckoutFlow.jsx` payment failure) and silent `console.error`-only catch blocks with `toast.error(...)`.

PHASE 5 — New Feature: Ad-hoc Extra Quantity Requests

Customer requests more milk for a specific date (e.g. event) → visible to admin and to the assigned delivery partner next morning, without touching the permanent subscription.

5.1 Schema

```sql
CREATE TABLE delivery_adjustments (
  id INT AUTO_INCREMENT PRIMARY KEY,
  customer_id INT NOT NULL,
  product_id INT NOT NULL,
  delivery_date DATE NOT NULL,
  extra_quantity DECIMAL(5,2) NOT NULL,
  note VARCHAR(255),
  status ENUM('PENDING','ACKNOWLEDGED','FULFILLED','CANCELLED') DEFAULT 'PENDING',
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (customer_id) REFERENCES customers(id) ON DELETE CASCADE,
  FOREIGN KEY (product_id) REFERENCES products(id) ON DELETE CASCADE
);
```

5.2 Backend — `adjustmentController.js`

```js
```

```
const pool = require('../config/db');

const create = async (req, res) => {
 const userId = req.user.id;
 const { product_id, delivery_date, extra_quantity, note } = req.body;

 // enforce cutoff: request must be for tomorrow or later, and made before 9 PM same day for next-day requests
 const requestDate = new Date(delivery_date);
 const now = new Date();
 const tomorrow = new Date(now); tomorrow.setDate(now.getDate() + 1); tomorrow.setHours(0,0,0,0);
 if (requestDate < tomorrow) {
  return res.status(400).json({ message: 'Requests must be for tomorrow or later' });
 }
 if (requestDate.toDateString() === tomorrow.toDateString() && now.getHours() >= 21) {
  return res.status(400).json({ message: 'Cutoff for tomorrow is 9 PM today' });
 }

 const [customerResult] = await pool.query('SELECT id FROM customers WHERE user_id = ?', [userId]);
 if (customerResult.length === 0) return res.status(404).json({ message: 'Customer not found' });
 const customerId = customerResult[0].id;

 await pool.query(
  `INSERT INTO delivery_adjustments (customer_id, product_id, delivery_date, extra_quantity, note)
   VALUES (?, ?, ?, ?, ?)`,
  [customerId, product_id, delivery_date, extra_quantity, note || null]
 );
 res.status(201).json({ message: 'Extra quantity request submitted' });
};

const listForAdmin = async (req, res) => {
 const date = req.query.date || new Date().toISOString().split('T')[0];
 const [rows] = await pool.query(`
  SELECT da.id, u.name as customer_name, p.name as product_name, da.extra_quantity, da.note, da.status
  FROM delivery_adjustments da
  JOIN customers c ON da.customer_id = c.id
  JOIN users u ON c.user_id = u.id
  JOIN products p ON da.product_id = p.id
  WHERE da.delivery_date = ?
 `, [date]);
 res.json(rows);
};

module.exports = { create, listForAdmin };
```

Routes:

`customerRoutes.js`: `router.post('/adjustments', create);`
`adminRoutes.js`: `router.get('/adjustments', listForAdmin);`

5.3 Delivery boy route — join adjustments in

Modify `deliveryController.getTodayRoute` (already touched in Phase 0.2) to also LEFT JOIN today's adjustment:

```
js
LEFT JOIN delivery_adjustments da ON da.customer_id = c.id AND da.delivery_date = CURDATE() AND da.status != 'CANCELLED'
```

and select `da.extra_quantity, da.product_id as extra_product_id, da.note as extra_note`. Frontend (`DeliveryDashboard.jsx`) renders a chip when `stop.extra_quantity` is present:

```jsx
{stop.extra_quantity > 0 && (
 <span className="ml-2 text-[10px] bg-orange-100 text-orange-700 px-2 py-0.5 rounded-full font-bold">
  🎉 +{stop.extra_quantity} today{stop.extra_note ? ` — ${stop.extra_note}` : ''}
 </span>
)}
```

5.4 Fulfillment — no immediate billing, accumulate for month-end

Decision: daily/subscription customers get extras folded into the regular monthly bill, not charged instantly. `markDelivered` only settles the adjustment's status — it does **not** touch `bills`:

```js
// inside markDelivered, after the deliveries INSERT ... ON DUPLICATE KEY UPDATE above
if (delivered) {
 await pool.query(
  `UPDATE delivery_adjustments SET status='FULFILLED'
   WHERE customer_id=? AND delivery_date=CURDATE() AND status='PENDING'`,
  [customerId]
 );
}
```

If `delivered` is false (customer marked not-available / missed), leave the adjustment `PENDING` — it wasn't actually fulfilled, don't bill for it, and don't silently drop it either (admin can see it's still outstanding in `getAdjustments`).

`generateBills` **must now roll extras into the monthly total.** Add a second correlated subquery alongside the existing `daily_cost` one, summing fulfilled adjustments that fall inside the billing month, and add it into `amount`:

```js
```

```
const [deliveries] = await pool.query(`
  SELECT
    d.customer_id,
    SUM(d.delivered_quantity) as total_liters,
    COUNT(d.id) as delivery_days,
    (
      SELECT SUM(cr.quantity * p.price_per_unit)
      FROM customer_requirements cr JOIN products p ON cr.product_id = p.id
      WHERE cr.customer_id = d.customer_id
    ) as daily_cost,
    (
      SELECT COALESCE(SUM(da.extra_quantity * p2.price_per_unit), 0)
      FROM delivery_adjustments da JOIN products p2 ON da.product_id = p2.id
      WHERE da.customer_id = d.customer_id
        AND da.status = 'FULFILLED'
        AND DATE_FORMAT(da.delivery_date, '%Y-%m') = CONCAT(?, '-', ?)
    ) as extra_cost
  FROM deliveries d
  WHERE d.status = 'DELIVERED' AND d.date LIKE ?
  GROUP BY d.customer_id
`, [year, monthNum, `${year}-${monthNum}-%`]);

// in the loop:
for (let del of deliveries) {
  const amount = (del.daily_cost * del.delivery_days) + Number(del.extra_cost);
  // ... existing insert/update into bills, unchanged bill_type='SUBSCRIPTION'
}
```

This means extras are invisible to the customer's `Bills.jsx` as a line item until month-end generation — worth surfacing on the *customer* side though: add a small note on `CustomerDashboard.jsx` or `Bills.jsx`, e.g. "Includes ₹X in extra requests this month," sourced from:

```
js
// customerController.getBills or a new lightweight endpoint
const [[{ pendingExtras }]] = await pool.query(`
 SELECT COALESCE(SUM(da.extra_quantity * p.price_per_unit),0) as pendingExtras
 FROM delivery_adjustments da JOIN products p ON da.product_id=p.id
 WHERE da.customer_id=? AND da.status='FULFILLED'
  AND MONTH(da.delivery_date)=MONTH(CURDATE()) AND YEAR(da.delivery_date)=YEAR(CURDATE())
`, [customerId]);
```

so they aren't surprised by the month-end total.

5.5 Admin visibility

`getDailyLogs` — add the same LEFT JOIN so `DailyLogs.jsx` can show a badge inline without a separate page:

```
sql
LEFT JOIN delivery_adjustments da ON da.customer_id = c.id AND da.delivery_date = d.date AND da.status != 'CANCELLED'
```

select `da.extra_quantity, da.note as extra_note`. In `DailyLogs.jsx`, render a badge next to `req` when `log.extra_quantity` is truthy.

5.6 Frontend — customer request form

Add to `CustomerDashboard.jsx`, a small card:

```jsx
const [showExtraModal, setShowExtraModal] = useState(false);
const [extraForm, setExtraForm] = useState({ product_id: '', delivery_date: '', extra_quantity: 1, note: '' });

const submitExtra = async () => {
 const token = localStorage.getItem('token');
 const res = await fetch('http://localhost:5000/api/customer/adjustments', {
  method: 'POST',
  headers: { 'Content-Type': 'application/json', 'Authorization': `Bearer ${token}` },
  body: JSON.stringify(extraForm)
 });
 if (res.ok) { toast.success('Request submitted!'); setShowExtraModal(false); }
 else { const err = await res.json(); toast.error(err.message); }
};
```

Date input `min` should default to tomorrow's ISO date to prevent invalid picks client-side (still validated server-side per 5.2).

5.7 Freshness — polling now, Socket.io before final delivery milestone

Decision: 45s polling is acceptable for this stage; plan to swap to Socket.io before the final release/delivery deadline, not immediately.

Now: `DailyLogs.jsx` and `DeliveryDashboard.jsx`: `useEffect(() => { const id = setInterval(fetchX, 45000); return () => clearInterval(id); }, [])`. Before final delivery: swap the polling `useEffect` for a socket listener (`adjustment:created` scoped to an admin room + the specific delivery-boy's room). Because the fetch functions (`fetchLogs`, `fetchRoute`) are already isolated, this is a drop-in swap — no other refactor needed, just replace the interval with `socket.on('adjustment:created', fetchLogs)`. Flag this explicitly as a pre-launch checklist item so it doesn't get missed.

Suggested Build Order

Phase 0 (bugs) — half day, unblocks everything else being tested against real data.
Phase 5 (adjustments feature) — since it's the newest customer-facing ask; do schema → backend → delivery route join → admin join → customer UI, in that order (each step is independently testable via Postman before wiring UI).
Phase 1 (Products CRUD, dashboard stats, collect payment) — core business gaps.
Phase 2 (axios client + validation + rate limiting) — do this refactor before Phase 1/5 UI grows further, so new code is written against the clean pattern instead of needing a second pass.
Phase 3 (transactions, cascades, migrations) — data-integrity pass once feature surface stabilizes.
Phase 4 (auth context, toasts) — polish, can be done incrementally alongside any phase above.

Decisions Locked In

Inventory: deferred — sidebar link stays dead, no backend work this pass.
Extra-quantity billing: accumulated into month-end `generateBills` run, not billed instantly. Customer sees a running "pending extras" total before the bill generates (5.4).
Real-time: 45s polling for now; swap to Socket.io as a pre-final-delivery checklist item (5.7). Fetch functions are already isolated so this is a low-effort swap when the time comes.